CSA1709 : Artificial Intelligence - SLOT : B

Lab Experiments

Name: Dillilokesh D
Register Number: 192311097

Experiment 1: 8-Puzzle Problem

Aim: To write a Python program to solve the 8-Puzzle problem using the Best First Search algorithm.

Algorithm:

1. Define the initial state and goal state.
2. Calculate the heuristic value (number of misplaced tiles).
3. Generate all possible moves (up, down, left, right).
4. Select the state with the minimum heuristic value.
5. Repeat until the goal state is reached.
6. Display the solution path.

Program:

```
from queue import PriorityQueue
goal = [[1,2,3],[4,5,6],[7,8,0]] def
heuristic(state):
   h = 0     for i in
range(3):         for j in
range(3):
          if state[i][j] != 0 and state[i][j] != goal[i][j]:
             h += 1
return h def
find_blank(state):     for
i in range(3):         for j
in range(3):             if
state[i][j] == 0:
return i, j def
copy(state):
```

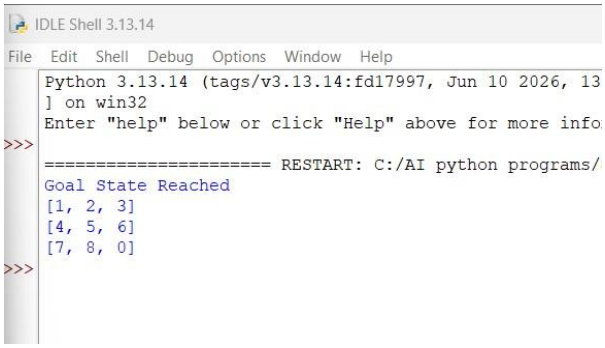
```
   return [row[:] for row in state] def
solve(start):
   pq = PriorityQueue()
pq.put((heuristic(start), start))
visited = []     while not
pq.empty():         h, state =
pq.get()         if state == goal:
            print("Goal State Reached")
for row in state:                  print(row)
return         visited.append(state)
x, y = find_blank(state)         moves =
[(-1,0),(1,0),(0,-1),(0,1)]         for dx,
dy in moves:              nx, ny = x+dx,
y+dy              if 0<=nx<3 and
0<=ny<3:
               new = copy(state)                  new[x][y],
new[nx][ny] = new[nx][ny], new[x][y]                  if new not
in visited:                     pq.put((heuristic(new), new)) start =
[[1,2,3],
           [4,0,6],
[7,5,8]] solve(start)
```

Sample Input:

Initial State

1 2 3
4 0 6
7 5 8

Output:



Result:

Thus, the Python program to solve the 8-Puzzle problem was executed successfully.

Experiment 2: 8-Queen Problem

Aim: To write a Python program to solve the 8-Queen problem using Backtracking.

Algorithm:

1. Place a queen in the first column.
2. Check whether the position is safe.
3. If safe, place the queen and move to the next column.
4. If no safe position exists, backtrack.
5. Continue until all eight queens are placed.
6. Display the solution.

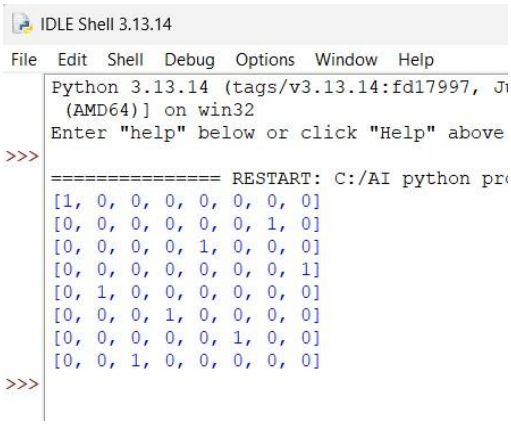
Program:

```
N = 8 board = [[0]*N for _ in
range(N)] def safe(row,col):
for i in range(col):         if
board[row][i]:              return
False     i,j=row,col     while i>=0
and j>=0:         if board[i][j]:
return False         i-=1         j-=1
i,j=row,col     while i<N and
j>=0:         if board[i][j]:
return False         i+=1         j-=1
return True def solve(col):     if
col>=N:         return True     for i
in range(N):
       if safe(i,col):
           board[i][col]=1
if solve(col+1):
return True
board[i][col]=0
return False solve(0)
for row in board:
   print(row)
```

Sample Input:

N = 8

Output:



Result:

Thus, the Python program to solve the 8-Queen problem was executed successfully.

Experiment 3: Water Jug Problem

Aim: To write a Python program to solve the Water Jug problem.

Algorithm:

1. Define the capacities of the two jugs.
2. Fill the larger jug.
3. Pour water into the smaller jug.
4. Empty the smaller jug whenever it becomes full.
5. Repeat the process until the required quantity is obtained.
6. Display each step.

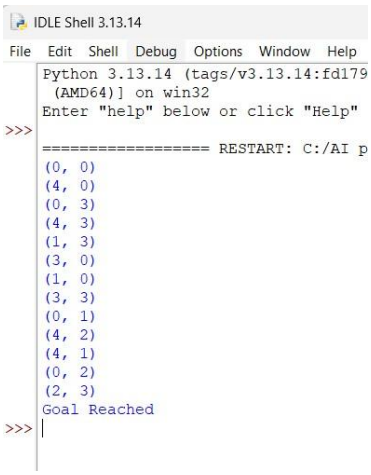
Program:

from collections import deque

```
visited=set()
queue=deque()
queue.append((0,0)) while
queue:
x,y=queue.popleft()     if
(x,y) in visited:
continue
visited.add((x,y))
print((x,y))     if x==2:
print("Goal Reached")
        break
possible=[
        (4,y),
        (x,3),
        (0,y),
        (x,0),
        (max(0,x-(3-y)),min(3,x+y)),
        (min(4,x+y),max(0,y-(4-x)))
    ]
    for state in possible:
if state not in visited:
queue.append(state)
```

Sample Input:

Jug1 Capacity = 4, Jug2 Capacity = 3, Target = 2 Output:



Result:

Thus, the Python program to solve the Water Jug problem was executed successfully.

Experiment 4: Crypt-Arithmetic Problem

Aim: To write a Python program to solve the Crypt-Arithmetic problem.

Algorithm:

1. Consider the words SEND + MORE = MONEY.
2. Assign a unique digit (0–9) to each letter.
3. Ensure no two letters have the same digit.
4. Check whether the arithmetic equation is satisfied.
5. If satisfied, display the solution.
6. Stop after finding the valid solution.

Program:

```
from itertools import permutations letters='SENDMORY'

for p in permutations(range(10),8):
```

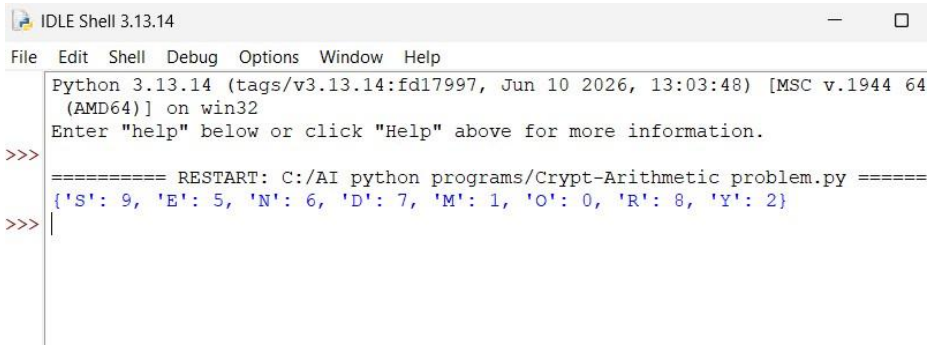
```
    d=dict(zip(letters,p))      if
d['S']==0 or d['M']==0:
        continue      send=1000*d['S']+100*d['E']+10*d['N']+d['D']
more=1000*d['M']+100*d['O']+10*d['R']+d['E']
money=10000*d['M']+1000*d['O']+100*d['N']+10*d['E']+d['Y']
if send+more==money:
        print(d)
```

break Sample

Input:

SEND + MORE = MONEY

Output:



Result

Thus, the Python program to solve the Crypt-Arithmetic problem was executed successfully.

Experiment 5: Missionaries and Cannibals Problem

Aim: To write a Python program to solve the Missionaries and Cannibals problem using Breadth First Search (BFS).

Algorithm:

1. Represent the state as (Missionaries, Cannibals, Boat).
2. Start from the initial state (3,3,Left).
3. Generate all possible valid moves.
4. Discard invalid states where missionaries are outnumbered.
5. Continue searching using BFS.
6. Stop when the goal state (0,0,Right) is reached.
7. Display the sequence of states.

Program: from collections

import deque start=(3,3,1)

goal=(0,0,0)

queue=deque([start])

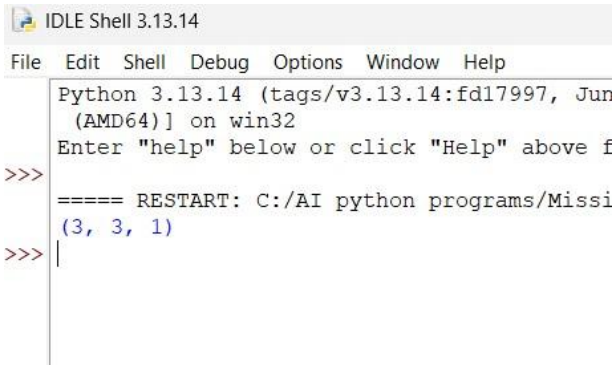
visited=set() while queue:

```
    state=queue.popleft()
if state==goal:
        print("Goal Reached")
break     if state in visited:
        continue
    visited.add(state)
```

print(state) Sample

Input: No input

Output:



Result:

Thus, the Python program to solve the Missionaries and Cannibals problem was executed successfully.

Experiment 6: Vacuum Cleaner Problem

Aim: To write a Python program to implement the Vacuum Cleaner problem.

Algorithm:

1. Start with the vacuum cleaner in Room A.
2. Check whether the current room is dirty.
3. If dirty, clean the room.
4. Move to the next room.
5. Repeat the process until all rooms are clean.
6. Display the cleaning status.

Program: rooms

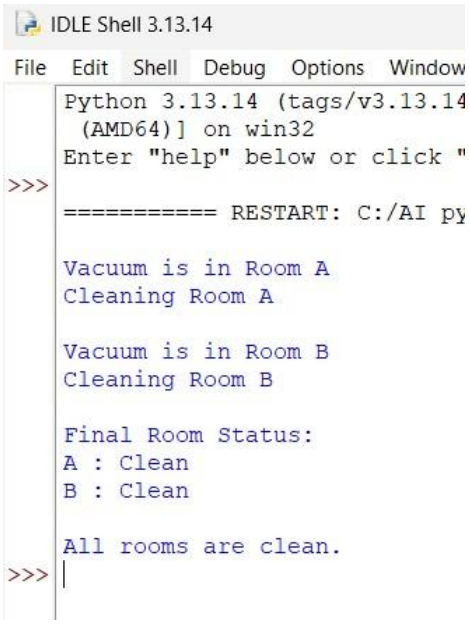
```
= {

   'A': 'Dirty',

   'B': 'Dirty'

} location =

'A' while

True:

   print("\nVacuum is in Room", location)

if rooms[location] == 'Dirty':

print("Cleaning Room", location)

rooms[location] = 'Clean'     else:
```

```
        print("Room", location, "is already Clean")
if location == 'A':         location = 'B'
    else:
break
print("\nFinal Room Status:") for
room in rooms:
    print(room, ":", rooms[room]) print("\nAll
rooms are clean.")
```

Sample Input:

Room A = Dirty

Room B = Dirty Output:



Result:

Thus, the Python program to implement the Vacuum Cleaner problem was executed successfully.

Experiment 7: Breadth First Search (BFS)

Aim: To write a Python program to implement Breadth First Search (BFS).

Algorithm:

1. Create a graph using an adjacency list.
2. Select the starting node.
3. Mark the starting node as visited.
4. Insert the starting node into the queue.
5. Remove one node from the queue.
6. Visit all unvisited adjacent nodes and insert them into the queue.
7. Repeat until the queue becomes empty.
8. Display the BFS traversal.

Program: from collections

```
import deque graph = {

   'A': ['B', 'C'],

   'B': ['D', 'E'],

   'C': ['F'],

   'D': [],

   'E': ['F'],

   'F': []

} visited = set()

queue = deque()

start = 'A'
visited.add(start)

queue.append(start)
```

print("BFS Traversal:") while

queue:

node = queue.popleft()

print(node, end=" ") for

neighbor in graph[node]:

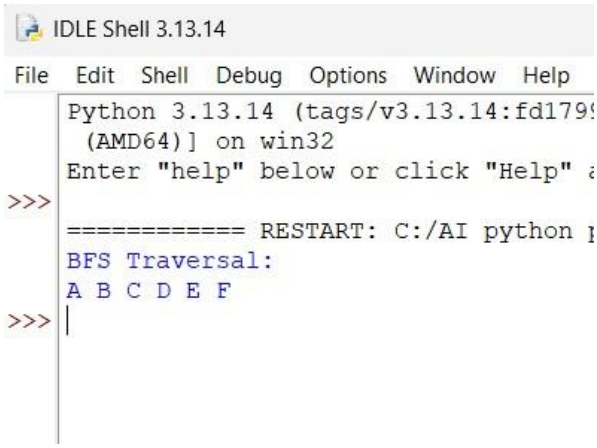
if neighbor not in visited:

visited.add(neighbor)

queue.append(neighbor)

Sample Input: Starting

Node = A Output:



Result:

Thus, the Python program to implement Breadth First Search (BFS) was executed successfully.

Experiment 8: Depth First Search (DFS)

Aim : To write a python program to implement the Depth First Search (DFS)

Algorithm :

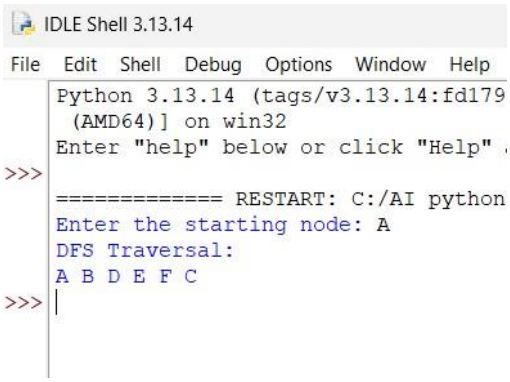
1. Define the graph using an adjacency list.
2. Create an empty set to store the visited nodes.
3. Start the traversal from the given source node.
4. Mark the current node as visited and display it.
5. Visit each adjacent node recursively if it has not been visited.
6. Continue the process until all reachable nodes are visited.
7. End the traversal when no unvisited adjacent nodes remain.

Program : graph

```
= {
   'A': ['B', 'C'],
   'B': ['D', 'E'],
   'C': ['F'],
   'D': [],
   'E': ['F'],
   'F': []
} visited =
set() def
dfs(node):
   if node not in visited:
print(node, end=" ")
visited.add(node)        for
neighbour in graph[node]:
          dfs(neighbour) start =
input("Enter the starting node: ") if
start in graph:
   print("DFS Traversal:")
   dfs(start) else:
print("Invalid starting node.")
```

Sample Input:

Enter the starting node: A Output:



Result:

Thus, the Depth First Search (DFS) algorithm was implemented successfully in Python, and the graph was traversed in depth-first order starting from the specified source node.

Experiment 9: Travelling Salesman Problem

Aim: To write a python program to implement the Travelling Salesman Problem (TSP).

Algorithm:

1. Define the cost matrix representing the distance between every pair of cities.
2. Consider the first city as the starting city.
3. Generate all possible permutations of the remaining cities.
4. Calculate the total travel cost for each possible route.
5. Compare the costs of all routes.
6. Store the route with the minimum travel cost.
7. Display the optimal route and its minimum cost.

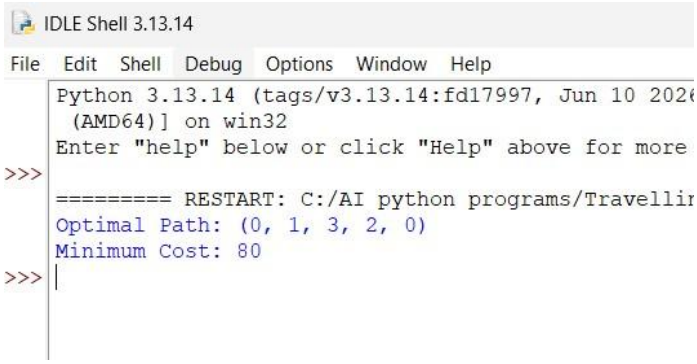
Program: from itertools import
permutations graph = [
[0, 10, 15, 20],
[10, 0, 35, 25],
[15, 35, 0, 30],
[20, 25, 30, 0]
]
n = len(graph) cities =
list(range(1, n)) min_cost =
float('inf') best_path = None for
path in permutations(cities):
current_cost = 0
current_city = 0 for
city in path:
current_cost += graph[current_city][city]
current_city = city current_cost
+= graph[current_city][0] if
current_cost < min_cost: min_cost
= current_cost best_path = (0,) +
path + (0,) print("Optimal Path:",
best_path) print("Minimum Cost:",
min_cost)

Sample Input:

Cost Matrix

0 10 15 20
10 0 35 25
15 35 0 30

20 25 30 0 Output:



Result:

Thus, the Travelling Salesman Problem (TSP) was successfully implemented in Python using the brute-force permutation method

Experiment 10: Program to Implement A* Algorithm

Aim: To write a python program to implement the A* (A-Star) search algorithm

Algorithm:

1. Define the graph with edge costs and heuristic values.
2. Create an open list containing the start node.
3. Create an empty closed list.
4. Select the node with the lowest f(n) = g(n) + h(n) value.
5. If the selected node is the goal node, stop the search.

6. Otherwise, expand all neighboring nodes.
7. Calculate the cost and update the parent of each neighbor if a better path is found.
8. Repeat until the goal node is reached or no path exists.
9. Display the shortest path and its total cost.

Program: graph

```
= {

   'A': [('B', 1), ('C', 3)],

   'B': [('D', 3), ('E', 6)],

   'C': [('F', 5)],

   'D': [('G', 2)],

   'E': [('G', 1)],

   'F': [('G', 2)],

   'G': [] }

heuristic = {

   'A': 7,

   'B': 6,
   'C': 4,

   'D': 2,

   'E': 1,

   'F': 3,

   'G': 0 } def

a_star(start, goal):

open_list = {start}

closed_list = set()     g =
```

```
{start: 0}     parents =
{start: start}     while
open_list:         n =
None         for v in
open_list:
            if n is None or g[v] + heuristic[v] < g[n] + heuristic[n]:
                n = v         if n ==
goal:              path = []
while parents[n] != n:
path.append(n)                 n =
parents[n]
path.append(start)
path.reverse()
print("Shortest Path:", path)
print("Total Cost:", g[goal])
return         for (m, weight)
in graph[n]:              if m not
in open_list and m not in
closed_list:
                open_list.add(m)
parents[m] = n                 g[m]
= g[n] + weight
```

else: if g[m] > g[n]

+ weight: g[m] = g[n] +

weight parents[m] = n

if m in closed_list:

closed_list.remove(m)

open_list.add(m)

open_list.remove(n)

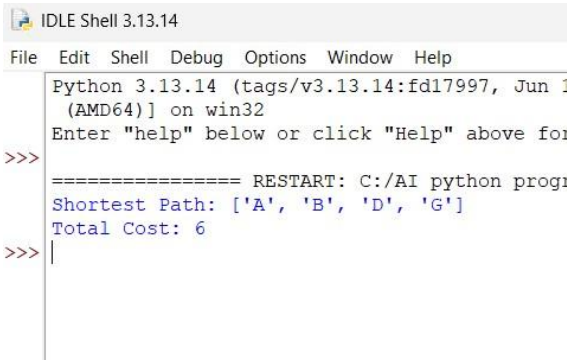
closed_list.add(n) print("Path does

not exist.") a_star('A', 'G')

Sample Input:

Start Node : A
Goal Node : G

Output:



Experiment11: Map Coloring using Constraint Satisfaction Problem (CSP)

Aim: To write a python program to implement the Map Coloring Problem using the Constraint Satisfaction Problem (CSP)

Algorithm:

1. Define the map with neighboring regions.
2. Define the available colors.
3. Assign colors to each region one by one.
4. Check whether the assigned color satisfies the constraints.
5. If the color is valid, proceed to the next region.
6. Otherwise, try another color.
7. If no color is suitable, backtrack.
8. Continue until all regions are successfully colored.
9. Display the final color assignment.

Program:

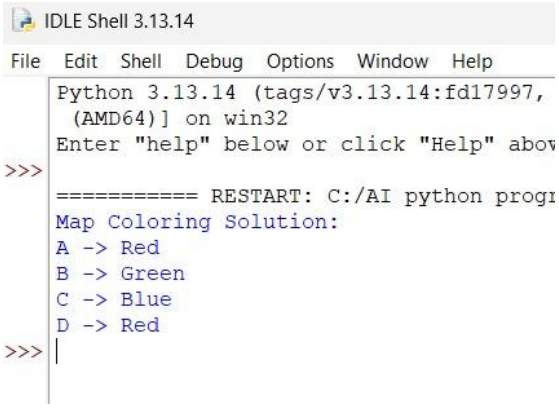
```
states = ['A', 'B', 'C', 'D'] neighbors
= {
```

```
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D'],
    'D': ['B', 'C']
}
colors = ['Red', 'Green', 'Blue']
color_map = {} def is_safe(state, color):     for neighbor in
neighbors[state]:         if neighbor in color_map and
color_map[neighbor] == color:             return False     return True
def solve(index):     if index == len(states):         return True
state = states[index]     for color in colors:         if is_safe(state,
color):
            color_map[state] = color
if solve(index + 1):
return True             del
color_map[state]     return False
if solve(0):
    print("Map Coloring Solution:")
for state in states:         print(state, "->",
color_map[state]) else:     print("No
solution exists.")
```

Sample Input:

States:
A, B, C, D Available
Colors:
Red, Green, Blue

Output:



Experiment 12: Tic Tac Toe Game

Aim:

To implement a Tic Tac Toe Game in Python for two players, allowing players to place their symbols alternately until one player wins or the game ends in a draw.

Algorithm:

1. Create a 3×3 game board.
2. Display the empty board.
3. Allow Player X and Player O to enter their positions alternately.
4. Check whether the selected position is empty.
5. Update the board with the player's symbol.
6. After each move, check rows, columns, and diagonals for a winning condition.
7. If a player wins, display the winner and terminate the game.
8. If all positions are filled without a winner, declare the match as a draw.
9. End the program.

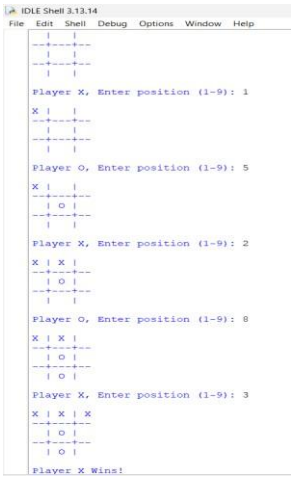
Program: board = [' ' for _
in range(9)] def display():

```
   print()
   print(board[0], "|", board[1], "|", board[2])     print("--+---+-
-")
   print(board[3], "|", board[4], "|", board[5])     print("--+---+-
-")
   print(board[6], "|", board[7], "|", board[8])
print() def winner(player):
   win = [(0,1,2),(3,4,5),(6,7,8),
         (0,3,6),(1,4,7),(2,5,8),            (0,4,8),(2,4,6)]
for x, y, z in win:         if board[x] == board[y] ==
board[z] == player:
            return True
return False player =
'X'
for move in range(9):
   display()     pos = int(input(f"Player {player}, Enter position
(1-9): ")) - 1     if board[pos] == ' ':          board[pos] = player
if winner(player):
            display()
            print("Player", player, "Wins!")
break          player = 'O' if player == 'X'
else 'X'     else:          print("Position
already occupied.") else:
   display()
print("Match Draw.")
```

Sample Input:

Player X : 1
Player O : 5
Player X : 2
Player O : 8
Player X : 3

Output:



Experiment 13: Minimax Algorithm for Gaming

Aim : To implement the Minimax algorithm in Python for a two-player game and determine the best possible move for a player by considering all possible game states.

Algorithm :

1. Create a game state or game tree.
2. Define terminal states with their utility values.
3. The Maximizer tries to obtain the maximum score.
4. The Minimizer tries to obtain the minimum score.
5. Recursively evaluate all possible moves.
6. Select the maximum value for the Maximizer.
7. Select the minimum value for the Minimizer.
8. Continue until the optimal move is obtained.
9. Display the best move and its score.

Program :

```
def minimax(depth, node, maximizing_player, tree):
if depth == 3:
    return tree[node]
if maximizing_player:
    best = -9999    for child in [2 * node, 2 *
node + 1]:        value = minimax(depth + 1, child,
False, tree)        best = max(best, value)
```

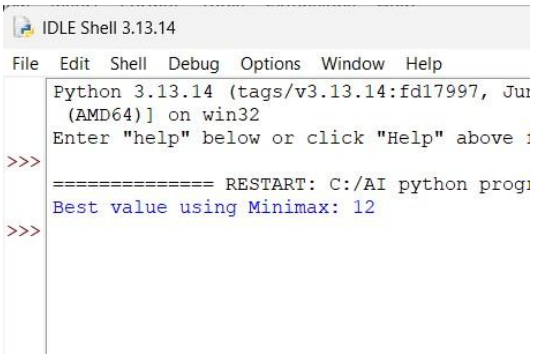
```
        return best
else:
        best = 9999         for child in [2 *
node, 2 * node + 1]:
            value = minimax(depth + 1, child, True, tree)
            best = min(best, value)
        return best tree
= {
    8: 3,
    9: 5,
    10: 2,
    11: 9,
    12: 12,
    13: 5,
    14: 23,
    15: 23 }
best_value = minimax(0, 1, True, tree) print("Best
value using Minimax:", best_value)
```

Sample Input:

Game Tree Leaf Values:

3 5 2 9 12 5 23 23

Output :



Result:

Thus, the Minimax algorithm was successfully implemented in Python to determine the best possible outcome for the maximizing player in a two-player game.

Experiment 14. Alpha-Beta Pruning Algorithm for Gaming

Aim: To implement the Alpha-Beta Pruning algorithm in Python to optimize the Minimax search by eliminating branches that cannot affect the final decision.

Algorithm:

1. Start with the root node of the game tree.
2. Initialize Alpha = -∞ and Beta = +∞.
3. Alpha represents the best value found for the Maximizer.
4. Beta represents the best value found for the Minimizer.
5. Explore the game tree recursively.
6. For Maximizer, update Alpha with the maximum value.
7. For Minimizer, update Beta with the minimum value.
8. If Beta ≤ Alpha, stop exploring the remaining branches.
9. Return the optimal value.
10. Display the best value obtained.

Program: def alpha_beta(depth, node,

maximizing_player,

alpha, beta, tree):

if depth == 3:

return tree[node]

if maximizing_player:

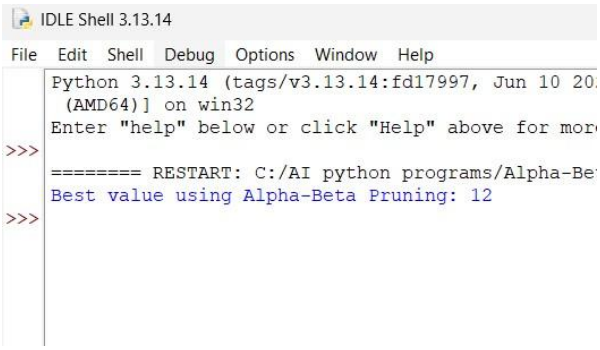
```
        best = -9999         for child in [2 *
node, 2 * node + 1]:
            value = alpha_beta(
depth + 1,                  child,
False,                  alpha,
                beta,
tree
            )
            best = max(best, value)
alpha = max(alpha, best)              if
beta <= alpha:
                break
return best     else:
        best = 9999         for child in [2 *
node, 2 * node + 1]:
            value = alpha_beta(
depth + 1,                  child,
True,                  alpha,
beta,                  tree
            )
```

```
            best = min(best, value)
beta = min(beta, best)              if
beta <= alpha:
                break
return best tree = {
    8: 3,
    9: 5,
    10: 2,
    11: 9,
    12: 12,
    13: 5,
    14: 23,
    15: 23 } alpha = -
9999 beta = 9999
best_value = alpha_beta(
    0, 1, True,
alpha, beta, tree
)
print("Best value using Alpha-Beta Pruning:", best_value)
```

Sample Input:

Game Tree Leaf Values:

3 5 2 9 12 5 23 23 Output:



Result:

Thus, the Alpha-Beta Pruning algorithm was successfully implemented in Python. It obtained the optimal value while reducing unnecessary exploration of the game tree.

Experiment 15. Decision Tree

Aim: To implement a Decision Tree classification algorithm in Python and classify data based on its features.

Algorithm:

1. Import the required libraries.
2. Create a dataset containing input features and target classes.
3. Separate the features and target values.
4. Create a Decision Tree Classifier.
5. Train the classifier using the dataset.
6. Provide a new input for prediction.
7. Predict the class of the new input.
8. Display the predicted result.

Program: data

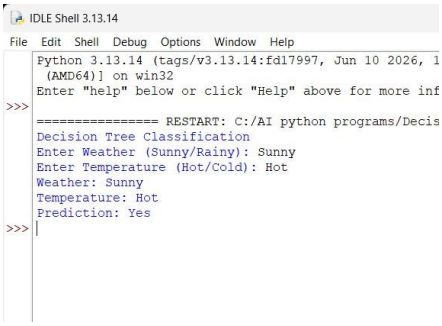
```
= [
   ["Sunny", "Hot", "Yes"],
   ["Sunny", "Cold", "Yes"],
   ["Rainy", "Hot", "No"],
   ["Rainy", "Cold", "No"],
   ["Sunny", "Hot", "Yes"],
   ["Rainy", "Cold", "No"]
]
def decision_tree(weather, temperature):
if weather == "Sunny":
      return "Yes"     elif
weather == "Rainy":
```

```
        return "No"
    else:
        return "Unknown" print("Decision Tree
Classification") weather = input("Enter Weather
(Sunny/Rainy): ") temperature = input("Enter
Temperature (Hot/Cold): ") result =
decision_tree(weather, temperature) print("Weather:",
weather) print("Temperature:", temperature)
print("Prediction:", result)
```

Sample Input :

Enter Weather (Rainy, Sunny): Sunny

Enter Temperature (Cold, Hot): Hot Output:



Result :

Thus, the Decision Tree classification algorithm was successfully implemented in Python and used to predict the class of a new data instance.

Experiment 16. Feed Forward Neural Network

Aim: To implement a Feed Forward Neural Network (FFNN) in Python for classification. The network passes input data forward through the hidden layer to the output layer without feedback connections.

Algorithm:

1. Import the required libraries.
2. Prepare the input dataset and target values.
3. Divide the dataset into training and testing data.
4. Create a feed-forward neural network.
5. Define the input, hidden, and output layers.
6. Train the neural network using the training data.
7. Test the trained model using test data.
8. Calculate the prediction accuracy.
9. Display the predicted and actual values.

Program:

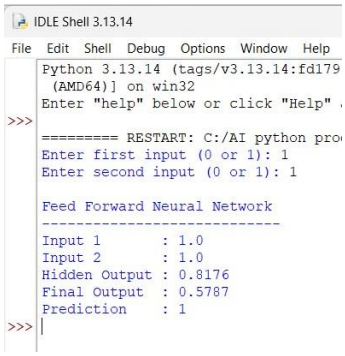
```
import math def

sigmoid(x):

    return 1 / (1 + math.exp(-x)) x1 =

float(input("Enter first input (0 or 1): ")) x2 =

float(input("Enter second input (0 or 1): ")) w1

= 0.5 w2 = 0.5 b1 = 0.5 hidden_input = (x1 *

w1) + (x2 * w2) + b1 hidden_output =

sigmoid(hidden_input) w3 = 1.0 b2 = -0.5

output_input = (hidden_output * w3) + b2

output = sigmoid(output_input) print("\nFeed

Forward Neural Network")
```

```
print("----------------------------") print("Input 1

:", x1) print("Input 2       :", x2) print("Hidden

Output :", round(hidden_output, 4)) print("Final

Output  :", round(output, 4)) if output >= 0.5:

    print("Prediction    : 1")

else:     print("Prediction

: 0")
```

Sample Input

Enter first input (0 or 1): 1 Enter

second input (0 or 1): 1 Output :



Result:

Thus, the Feed Forward Neural Network was successfully implemented in Python and trained to classify the given input data.

Experiment 17: Prolog Program to Sum the Integers from 1 to N

Aim: To write a Prolog program to find the sum of integers from 1 to N.

Algorithm:

1. Start the program.
2. Define a predicate sum(N, S) to calculate the sum of integers from 1 to N.
3. If N = 0, set the sum as 0.
4. Otherwise, recursively calculate the sum up to N-1.
5. Add N to the previous sum.
6. Display the result.
7. Stop.

Program:

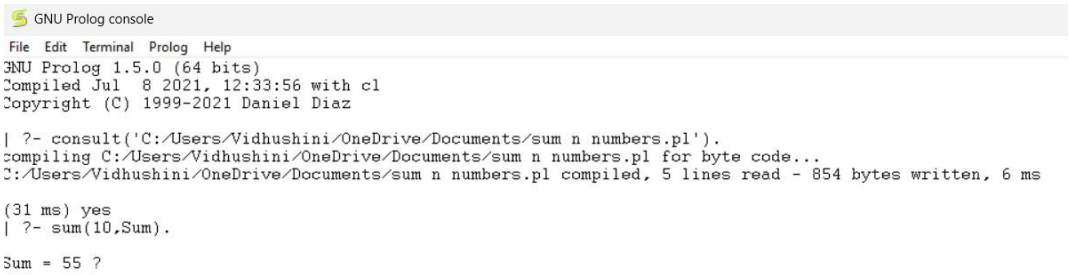
```
sum(0, 0). sum(N,
S) :-
   N > 0,
```

```
    N1 is N - 1,
sum(N1, S1),     S
is N + S1.
```

Query:

```
?- sum(10, S).
```

Output:



Result: The sum of integers from 1 to 10 is 55.

Experiment 18. Prolog Program for a Database with Name and DOB

Aim: To write a Prolog program to create a database containing Name and Date of Birth (DOB) and retrieve the DOB of a person.

Algorithm:

1. Start the program.
2. Define facts containing the person's name and date of birth.
3. Define a predicate dob(Name, Date) to retrieve the DOB.
4. Query the database using a person's name.
5. Display the corresponding DOB.
6. Stop.

Program:

```
dob(john, date(1990, 5, 1)).
dob(jane, date(1985, 12, 10)).
dob(bob, date(1978, 2, 28)).  dob(sue,
```

```
date(1995, 8, 15)).  dob(tom,
date(2000, 4, 22)).
lookup(Name, DOB) :-
dob(Name, DOB).
```

Query:

?- lookup(john, DOB).

Output:

```
(78 ms) yes
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/DOBs.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/DOBs.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/DOBs.pl compiled, 6 lines read - 1193 bytes written, 7 ms

(15 ms) yes
| ?- lookup(john,DOB).

DOB = date(1990,5,1)
```

Result: The DOB of the specified person is successfully retrieved from the database.

Experiment 19. Prolog Program for Student-Teacher-Sub-Code

Aim: To write a Prolog program to represent the relationship between students, teachers, and subjects/codes and retrieve the corresponding information.
Algorithm:

1. Start the program.
2. Define facts for students and their subjects.
3. Define facts for teachers and the subjects they handle.
4. Define subject codes.
5. Create a predicate to relate student, teacher, and subject code.
6. Query the database.
7. Display the result.
8. Stop.

Program:

teaches(john, math). teaches(jane, english).
teaches(bob, science). teaches(sue, history).
teaches(tom, art). takes(alice, math).
takes(alice, science). takes(bob, english).
takes(bob, science). takes(carol, history).
takes(carol, art). takes(dave, math).
takes(dave, english). takes(dave, art).
teaching_subjects(Teacher, Subject) :-
teaches(Teacher, Subject).
taking_students(Subject, Student) :-
takes(Student, Subject).

Query:

?- teaching_subjects(john, Subject).
?- taking_students(science, Student).
?- taking_students(math, Student), taking_students(art, Student).

Output

```
yes
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Stud-Teach.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Stud-Teach.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Stud-Teach.pl compiled, 15 lines read - 1912 bytes written, 11 ms

(15 ms) yes
| ?-  teaching_subjects(john, Subject).

Subject = math

yes
| ?-  taking_students(science, Student).

Student = alice ? ;

Student = bob ? .
Action (; for next solution, a for all solutions, RET to stop) ?

(47 ms) yes
| ?-  taking_students(math, Student), taking_students(art, Student).

Student = dave ? ;

(31 ms) no
```

Result: The student, teacher, and corresponding subject code are successfully obtained using Prolog relationships

Experiment 20. Prolog Program for Planets Database

Aim: To write a Prolog program to create a database of planets and retrieve information about them.

Algorithm:

1. Start the program.
2. Define facts containing the names of planets.
3. Store information such as order, type, and number of moons.
4. Define predicates to retrieve planet information.
5. Query the database.
6. Display the result.
7. Stop.

Program:

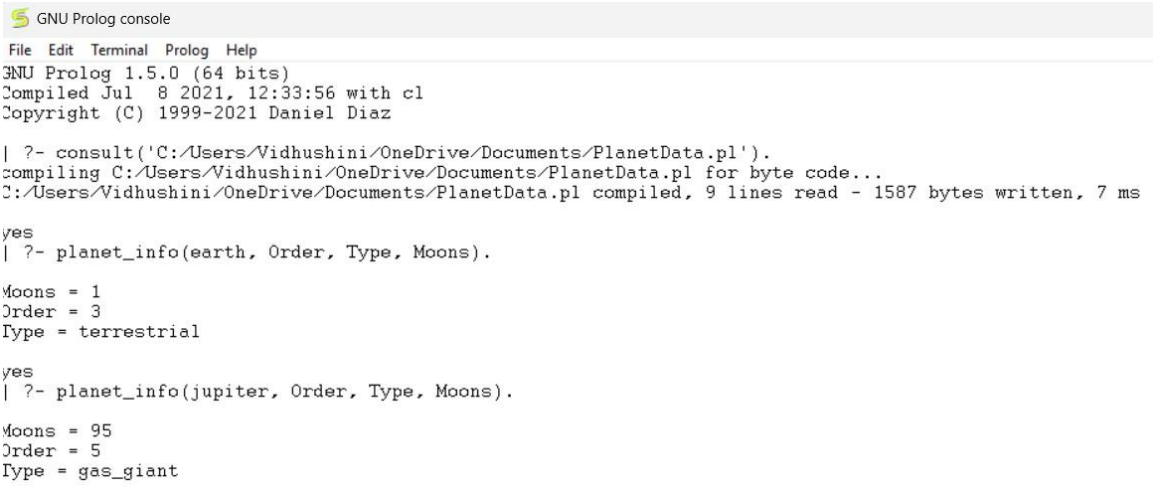
```
planet(mercury, 1, terrestrial, 0).
planet(venus, 2, terrestrial, 0).
planet(earth, 3, terrestrial, 1). planet(mars,
4, terrestrial, 2). planet(jupiter, 5,
gas_giant, 95). planet(saturn, 6,
gas_giant, 146). planet(uranus, 7,
ice_giant, 28). planet(neptune, 8,
ice_giant, 16).
```

planet_info(Name, Order, Type, Moons) :-
planet(Name, Order, Type, Moons).

Query:

?- planet_info(earth, Order, Type, Moons).
?- planet_info(jupiter, Order, Type, Moons).

Output:



Result: The planet database is successfully created and the required information is retrieved using Prolog.

Experiment 21. Prolog Program to Implement Towers of Hanoi

Aim

To write a Prolog program to implement the Towers of Hanoi problem using recursion.

Algorithm

1. Start with N disks on the source peg.
2. If there is only one disk, move it directly from source to destination.
3. Otherwise:
    - Move N-1 disks from source to auxiliary peg.
    - Move the largest disk from source to destination.
    - Move N-1 disks from auxiliary peg to destination.
4. Display all the required moves.
5. Stop.

```
Program hanoi(1, Source,

Destination, _) :-     write('Move

disk from '),     write(Source),

write(' to '),     write(Destination),

   nl.

hanoi(N, Source, Destination, Auxiliary) :-

   N > 1,
```

```
    N1 is N - 1,     hanoi(N1, Source,
Auxiliary, Destination),     write('Move disk
from '),     write(Source),
    write(' to '),
write(Destination),
    nl,
    hanoi(N1, Auxiliary, Destination, Source).
```

Query

```
?- hanoi(3, source, destination, auxiliary).
```

Output

```
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Tower of Hanoi.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Tower of Hanoi.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Tower of Hanoi.pl compiled, 15 lines read - 1604 bytes written, 7 ms

(16 ms) yes
| ?- hanoi(3, source, destination, auxiliary).
Move disk from source to destination
Move disk from source to auxiliary
Move disk from destination to auxiliary
Move disk from source to destination
Move disk from auxiliary to source
Move disk from auxiliary to destination
Move disk from source to destination

true ? |
```

Result

Thus, the Towers of Hanoi problem was successfully implemented using Prolog.

22. Prolog Program to Check Whether a Particular Bird Can Fly or Not

Aim

To write a Prolog program to determine whether a particular bird can fly or cannot fly using facts and rules.

Algorithm

1. Start the program.
2. Define facts for birds that can fly.
3. Define facts for birds that cannot fly.
4. Define the predicate can_fly(Bird).
5. Query the required bird.
6. Display whether the bird can fly or not.
7. Stop.

Program can_fly(eagle).

can_fly(parrot).

can_fly(pigeon).

can_fly(crow).

can_fly(sparrow).

cannot_fly(penguin).

cannot_fly(ostrich).

cannot_fly(emu).

bird_can_fly(Bird) :-

can_fly(Bird).

bird_cannot_fly(Bird) :-

cannot_fly(Bird).

Query 1

?- bird_can_fly(eagle).

?- bird_cannot_fly(penguin).

?- bird_can_fly(penguin).

Output:

```
(172 ms) yes
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Bird.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Bird.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Bird.pl compiled, 11 lines read - 1084 bytes written, 8 ms

(31 ms) yes
| ?- bird_can_fly(eagle).

yes
| ?- bird_cannot_fly(penguin).

yes
| ?- bird_can_fly(penguin).

no
| ?-
```

Result

Thus, the Prolog program successfully determines whether a particular bird can fly or not.

Experiment 23. Prolog Program to Implement Family Tree

Aim

To write a Prolog program to represent a family tree and determine relationships such as parent, grandparent, sibling, and child.

Algorithm

1. Start the program.
2. Define facts for parent-child relationships.
3. Define the father and mother relationships.
4. Define rules for grandparent.
5. Define rules for sibling.
6. Query the required relationship.
7. Display the result.
8. Stop.

Program father(ramesh,

rahul). father(ramesh,

priya). father(suresh,

anjali). mother(sita,

rahul). mother(sita,

priya). mother(geetha,

anjali).

parent(X, Y) :-

father(X, Y). parent(X,

Y) :- mother(X, Y).

grandparent(X, Z) :-

parent(X, Y),

parent(Y, Z). sibling(X,

Y) :- parent(P, X),

parent(P, Y),

X \= Y.

Query

?- parent(X, rahul).

?- sibling(rahul, X).

?- grandparent(X, rahul).

Output:

```
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Relation.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Relation.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Relation.pl compiled, 16 lines read - 1760 bytes written, 9 ms

yes
| ?- parent(X,rahul).

X = ramesh ? ;

X = sita ? .
Action (; for next solution, a for all solutions, RET to stop) ?

(15 ms) yes
| ?- sibling(rahul,X).

X = priya ? .
Action (; for next solution, a for all solutions, RET to stop) ?

(16 ms) yes
| ?- grandparent(X,rahul).

no
| ?- |
```

Result

Thus, the family tree was successfully implemented using Prolog facts and rules.

Experiment 24. Prolog Program to Suggest Dieting System Based on Disease

Aim: To write a Prolog program to suggest a suitable dieting system based on a person's disease.

Algorithm

1. Define diseases and recommended foods.
2. Define diseases and foods to avoid. 3. Create a predicate to suggest a diet.
4. Query the disease.
5. Display the recommended foods and foods to avoid.

Program recommended_diabetes(vegetables).

recommended_diabetes(whole_grains).

recommended_diabetes(fruits_in_moderation).

avoid_diabetes(sugary_foods).

avoid_diabetes(soft_drinks).

recommended_hypertension(fruits).

recommended_hypertension(vegetables).

recommended_hypertension(low_salt_food). avoid_hypertension(salty_foods).

avoid_hypertension(processed_foods). recommended_anemia(leafy_vegetables).

recommended_anemia(beetroot).

recommended_anemia(iron_rich_foods). avoid_anemia(excess_tea).

avoid_anemia(excess_coffee).

diet(Disease, Food) :- Disease

= diabetes,

recommended_diabetes(Food).

diet(Disease, Food) :- Disease =

hypertension,

recommended_hypertension(Food).

diet(Disease, Food) :- Disease

= anemia,

recommended_anemia(Food).

Query :

?- diet(diabetes, Food).

?- diet(hypertension, Food).

Output:

```
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Diet System.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Diet System.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Diet System.pl compiled, 23 lines read - 2467 bytes written, 11 ms

(15 ms) yes
| ?- diet(diabetes,Food).

Food = vegetables ? ;

Food = whole_grains ? ;

Food = fruits_in_moderation ? .
Action (; for next solution, a for all solutions, RET to stop) ?

yes
| ?- diet(hypertension,Food).

Food = fruits ? ;

Food = vegetables ? ;

Food = low_salt_food ? .
Action (; for next solution, a for all solutions, RET to stop) ?
```

Result: Thus, a Prolog-based diet suggestion system was successfully implemented based on disease.

Experiment 25. Prolog Program to Implement Monkey Banana Problem

Aim

To write a Prolog program to implement the Monkey Banana Problem, where a monkey uses a box to reach bananas placed at a height.

Algorithm

1. The monkey is initially on the floor.
2. The box is initially at another location.
3. The monkey moves to the box.
4. The monkey pushes the box below the bananas.
5. The monkey climbs onto the box.

6. The monkey grabs the bananas.
7. Display the sequence of actions.

```
Program move(state(middle, onfloor, middle,

hasnot),      state(middle, onfloor, middle,

hasnot)) :-     write('Monkey is already at the

middle.'), nl.

move(state(Pos1, onfloor, Pos1, Has),

state(Pos2, onfloor, Pos2, Has)) :-

Pos1 \= Pos2,     write('Monkey

moves from '),     write(Pos1),

write(' to '),     write(Pos2),

   nl.

push(state(Pos, onfloor, Pos, Has),
    state(middle, onfloor, middle, Has)) :-     Pos =

middle,     write('Monkey pushes the box to the

middle.'), nl.

climb(state(middle, onfloor, middle, Has),

state(middle, onbox, middle, Has)) :-     write('Monkey

climbs onto the box.'), nl. grab(state(middle, onbox,

middle, hasnot),      state(middle, onbox, middle, has)) :-

write('Monkey grabs the bananas.'), nl. solve :-

write('Monkey Banana Problem'), nl,     write('Monkey

moves to the box.'), nl,     write('Monkey pushes the box
```

below the bananas.'), nl, write('Monkey climbs onto

the box.'), nl, write('Monkey grabs the bananas.'), nl.

Query : ?- solve.

Output:

```
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Monkey Banana.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Monkey Banana.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Monkey Banana.pl compiled, 26 lines read - 4026 bytes written, 11 ms

yes
| ?- solve.
Monkey Banana Problem
Monkey moves to the box.
Monkey pushes the box below the bananas.
Monkey climbs onto the box.
Monkey grabs the bananas.

(16 ms) yes
| ?- |
```

Result : Thus, the Monkey Banana Problem was successfully implemented in Prolog.

Experiment 26. Prolog Program for Fruit and Its Color Using Backtracking

Aim: To write a Prolog program to find the color of different fruits using backtracking.

Algorithm

1. Start the program.
2. Define facts containing fruits and their colors.
3. Use the predicate fruit_color(Fruit, Color).
4. Query for a fruit or color.
5. Prolog uses backtracking to find all matching facts.
6. Display the results.
7. Stop.

Program :

fruit_color(apple, red). fruit_color(apple,

green). fruit_color(banana, yellow).

fruit_color(orange, orange).

fruit_color(grapes, green).

fruit_color(grapes, purple).

fruit_color(mango, yellow).

fruit_color(strawberry, red).

Query 1 – Find the color of apple

?- fruit_color(apple, Color).

?- fruit_color(Fruit, red).

Output :

```
(16 ms) yes
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Fruit color.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Fruit color.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Fruit color.pl compiled, 7 lines read - 1022 bytes written, 4 ms

(16 ms) yes
| ?- fruit_color(apple, Color).

Color = red ? ;

Color = green

(16 ms) yes
| ?- fruit_color(Fruit,red).

Fruit = apple ? ;

Fruit = strawberry
```

Result:

Thus, the fruit and color database using backtracking was successfully implemented in Prolog.

Experiment 27. Prolog Program to Implement Best First Search Algorithm

Aim

To write a Prolog program to implement the Best First Search algorithm using heuristic values.

Algorithm

1. Start from the initial node.
2. Store the nodes along with their heuristic values.
3. Select the node having the lowest heuristic value.
4. Expand the selected node.
5. Repeat the process until the goal node is reached.
6. Display the path from the start node to the goal node.

Program:

```
edge(a,b).

edge(a,c).

edge(b,d).

edge(b,e). edge(c,f).

edge(c,g).

edge(e,h).

edge(f,h).

heuristic(a,7). heuristic(b,6).

heuristic(c,5). heuristic(d,4).
```

```
heuristic(e,3). heuristic(f,2).

heuristic(g,6).

heuristic(h,0).

best_first(Start,Goal,Path) :-

best_search(Start,Goal,[Start],RevPath),     reverse(RevPath,Path).

best_search(Goal,Goal,Visited,Visited).

best_search(Current,Goal,Visited,Path) :-     findall(H-

Next,

          (edge(Current,Next),

\+ member(Next,Visited),

heuristic(Next,H)),

          List),     List \= [],

choose_best(List,Next),

best_search(Next,Goal,[Next|Visited],Path).

choose_best([_-N],N).

choose_best([H1-N1,H2-N2|Rest],Best) :-

H1 =< H2,     choose_best([H1-

N1|Rest],Best). choose_best([H1-N1,H2-

N2|Rest],Best) :-     H1 > H2,

choose_best([H2-N2|Rest],Best).
```

Query:

```
?- best_first(a, h, Path).
```

Output

```
yes
| ?- best_first(a,h,Path).

Path = [a,c,f,h] ? .
Action (; for next solution, a for all solutions, RET to stop) ? .
Action (; for next solution, a for all solutions, RET to stop) ? ;
```

Result

Thus, the Best First Search algorithm was successfully implemented using Prolog.

Experiment 28. Prolog Program for Medical Diagnosis

Aim

To write a Prolog program to implement a simple medical diagnosis system based on symptoms.

Algorithm

1. Start the program.

2. Define symptoms for different diseases.
3. Define rules connecting symptoms with diseases.
4. Query the symptoms of a patient.
5. Use the rules to identify the possible disease.
6. Display the diagnosis.
7. Stop.

```
Program symptom(ravi,

fever). symptom(ravi,

cough). symptom(ravi,

body_pain).

symptom(priya, fever).

symptom(priya, sneezing).

symptom(priya, running_nose). symptom(anu,

stomach_pain).

symptom(anu, vomiting).

diagnosis(Person, flu) :-

symptom(Person, fever),

symptom(Person, cough),

symptom(Person, body_pain).

diagnosis(Person, common_cold) :-

symptom(Person, fever),

symptom(Person, sneezing),

symptom(Person, running_nose).

diagnosis(Person, food_poisoning) :-
```

symptom(Person, stomach_pain),

symptom(Person, vomiting).

Query 1

?- diagnosis(ravi, Disease).

Output

```
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Medical Diagnosis.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Medical Diagnosis.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Medical Diagnosis.pl compiled, 18 lines read - 2225 bytes written, 7 ms

yes
| ?- diagnosis(ravi,Disease).

Disease = flu ? .
Action (; for next solution, a for all solutions, RET to stop) ?
```

Result

Thus, a simple medical diagnosis system was successfully implemented using Prolog.

Experiment 29. Prolog Program for Forward Chaining

Aim

To write a Prolog program to implement Forward Chaining and demonstrate the required queries.

Algorithm

1. Start with known facts.

2. Check the rules whose conditions match the known facts.
3. Apply the matching rules.
4. Add the newly derived facts to the knowledge base.
5. Continue until the required conclusion is obtained or no new fact can be derived.
6. Display the result.

Program fact(sunny).

fact(weekend).

rule(go_out) :-

fact(sunny),

fact(weekend).

rule(play_cricket) :-

fact(go_out). derive(X)

:- rule(X),

assertz(fact(X)),

write(X), write(' is

derived.'), nl.

Query 1

?- rule(go_out).

Output

```
(78 ms) yes
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Forward chaining.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Forward chaining.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Forward chaining.pl compiled, 11 lines read - 1215 bytes written, 6 ms

yes
| ?- rule(go_out).

yes
```

Result

Thus, the Forward Chaining concept was successfully implemented in Prolog by deriving new facts from existing facts and rules.

Experiment 30. Prolog Program for Backward Chaining

Aim

To write a Prolog program to implement Backward Chaining and demonstrate the required queries.

Algorithm

1. Start with a goal.
2. Check whether the goal is a known fact.
3. If it is not a fact, find a rule whose conclusion matches the goal.
4. Treat the rule's conditions as sub-goals.
5. Recursively prove each sub-goal.

6. If all sub-goals are true, the original goal is true.
7. Display the result.

Program

```
rain. cloudy.

wet_ground :-

   rain.

carry_umbrella :-

   rain,     cloudy.

go_out :-

carry_umbrella.
```

Query 1 ?-

```
wet_ground.

?- carry_umbrella.
?- go_out.
```

Output

```
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Backward chaining.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Backward chaining.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Backward chaining.pl compiled, 8 lines read - 796 bytes written, 6 ms

yes
| ?- wet_ground.

yes
| ?- carry_umbrella.

yes
| ?- go_out.

yes
| ?- |
```

Result

Thus, the Backward Chaining mechanism was successfully demonstrated by starting from the goal and checking the required conditions.

Experiment 31.Web Blog Using WordPress – Anchor Tag and Title Tag

Aim

To create a simple web blog demonstrating Title Tag, Heading Tag, Paragraph Tag, Anchor Tag, Image Tag, and List Tag.

Algorithm

1. Create a basic HTML webpage structure.
2. Add a title using the <title> tag.
3. Add blog headings using <h1> and <h2>.
4. Add text using <p> and links using <a>.
5. Add an image and list of blog topics.
6. Open the HTML page in a browser and verify all elements.

Program

```
<!DOCTYPE html>
<html>
<head>
<title>My AI Technology Blog</title>
</head>
<body>
<h1>Welcome to My AI Technology Blog</h1>
<h2>Artificial Intelligence</h2>
```

Artificial Intelligence enables computers to perform tasks that normally require human intelligence.

<h2>Useful Links</h2>

 Visit Google

 Visit Wikipedia

<h2>Blog Topics</h2>

Artificial Intelligence

Machine Learning

Deep Learning

Robotics

<h2>About the Blog</h2>

This blog provides basic information about modern AI and technology.

</p>

</body>

</html>

Result

The web blog was successfully created with Title, Heading, Paragraph, Anchor, and List tags. The hyperlinks can be clicked to navigate to the specified websites